# Exercise series 9: Concurrency Control



ADReM
Advanced Database Research & Modelling
University of Antwerp

Universiteit Antwerpen

# Problem 1:

Is this schedule:

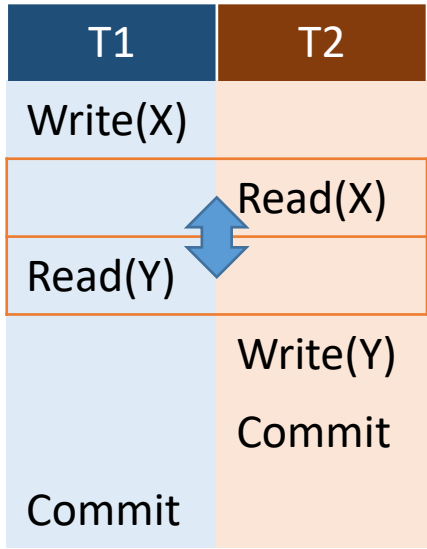
- *Conflict-serializable?*
- *Serializable?*
- *Recoverable?*
- *Cascadeless?*

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 1:

Is this schedule:

- *Conflict-serializable?*
- *Serializable?*
- *Recoverable?*
- *Cascadeless?*



# Problem 1:

Is this schedule:

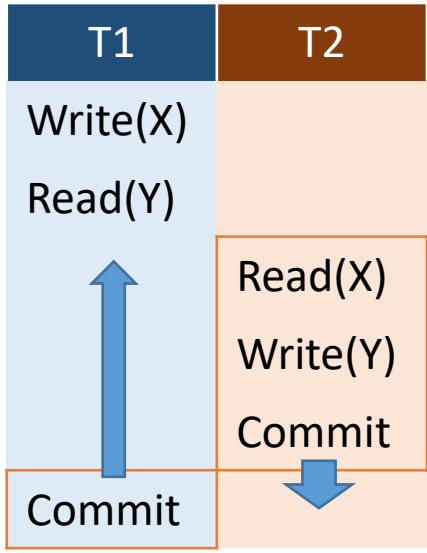
- *Conflict-serializable?*
- *Serializable?*
- *Recoverable?*
- *Cascadeless?*

| T1 | T2 |
|---|---|
| Write(X) | |
| Read(Y) | |
| | Read(X) |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 1:

Is this schedule:

- *Conflict-serializable?*
- *Serializable?*
- *Recoverable?*
- *Cascadeless?*



# Problem 1:

Is this schedule:

- *Conflict-serializable: Yes!*
- *Serializable?*
- *Recoverable?*
- *Cascadeless?*

| T1 | T2 |
|---|---|
| Write(X) | |
| Read(Y) | |
| Commit | |
| | Read(X) |
| | Write(Y) |
| | Commit |

# Problem 1:

Is this schedule:

- *Conflict-serializable: Yes!*
- *Serializable: Yes!*
- *Recoverable?*
- *Cascadeless?*

*Conflict-serializable → Serializable*

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 1:

Is this schedule:

- *Conflict-serializable: Yes!*
- *Serializable: Yes!*
- *Recoverable: No!*
- *Cascadeless?*

Formally: there exists a read of T2 that depends on a write of T1 BUT T2 commits before T1

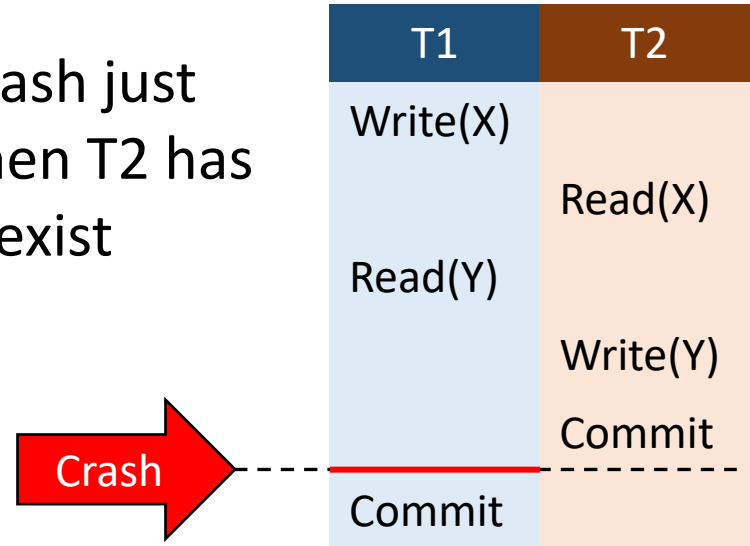
| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 1:

Is this schedule:

- *Conflict-serializable: Yes!*
- *Serializable: Yes!*
- *Recoverable: No!*
- *Cascadeless?*

Informally: if we have a crash just after T2 has committed then T2 has read a value that doesn't exist anymore.

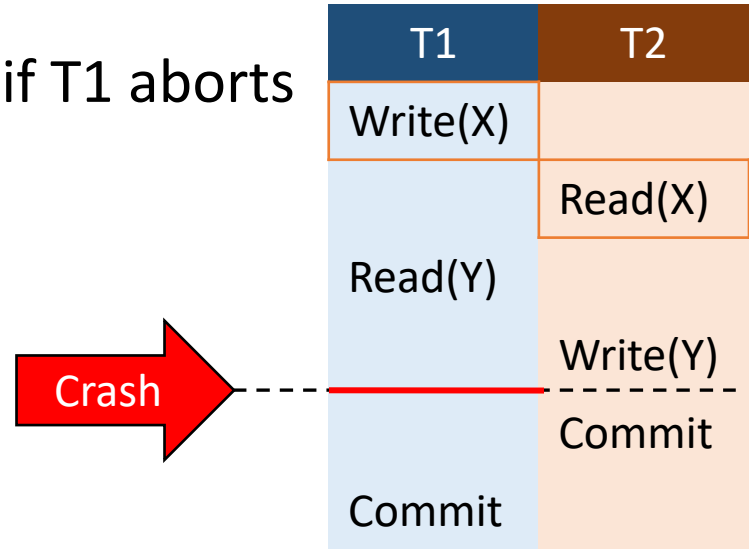


# Problem 1:

Is this schedule:

- *Conflict-serializable: Yes!*
- *Serializable: Yes!*
- *Recoverable: No!*
- *Cascadeless: No!*

T2 has a dirty read on T1, if T1 aborts T2 must do the same.



# Problem 2:

Can this schedule be expressed with

- *2PL?*
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL?*
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| T1 | T2 |
|---|---|
| | |
| | |
| Write(X) | |
| Unlock(X) | |
| | S-lock(X) |
| | Read(X) |
| Read(Y) | |
| Unlock(Y) | |
| | X-lock(Y) |
| | Write(Y) |
| | |
| | |
| | Commit |
| Commit | |

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| T1 | T2 |
|---|---|
| X-lock(X) | |
| S-lock(Y) | |
| Write(X) | |
| Unlock(X) | |
| | S-lock(X) |
| | Read(X) |
| Read(Y) | |
| Unlock(Y) | |
| | X-lock(Y) |
| | Write(Y) |
| | Unlock(X) |
| | Unlock(Y) |
| | Commit |
| Commit | |

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL?*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

**CONFLICT!**

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

**CONFLICT!**

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 0 | 0 | 0 | 0 |

→

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 0 | 1 | 0 | 0 |

→

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 2 | 1 | 0 | 0 |

→

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 2 | 1 | 1 | 0 |

| | T1 | T2 |
|---|---|---|
| | Write(X) | |
| | | Read(X) |
| → | Read(Y) | |
| | | Write(Y) |
| | | Commit |
| | Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 2 | 1 | 1 | 2 |

→

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule?*
- *Timestamp* with *Thomas Write Rule?*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 2 | 1 | 1 | 2 |

→

| T1 | T2 |
|---|---|
| Write(X) | |
| | Read(X) |
| Read(Y) | |
| | Write(Y) |
| | Commit |
| Commit | |

# Problem 2:

Can this schedule be expressed with

- *2PL: Yes!*
- *Strict 2PL: No!*
- *Rigorous 2PL: No!*
- *Timestamp* without *Thomas Write Rule: Yes!*
- *Timestamp* with *Thomas Write Rule: Yes!*

| X | | Y | |
|---|---|---|---|
| R | W | R | W |
| 2 | 1 | 1 | 2 |

| | T1 | T2 |
|---|---|---|
| | Write(X) | |
| | | Read(X) |
| | Read(Y) | |
| | | Write(Y) |
| | | Commit |
| → | Commit | |